

A horizontal bar with a blue segment on the left and a green segment on the right.

api.goexpressparaguay.com

Guía del integrador

API pública de GO EXPRESS para crear envíos, consultar estados, cotizar rutas y recibir webhooks de cada cambio de estado. De cero a tu primer envío en menos de 5 minutos.

VERSIÓN DEL API

v1

FECHA

Agosto 2026

BASE URL

<https://api.goexpressparaguay.com>

A horizontal bar with a blue segment on the left and a green segment on the right.

Contenido

1	Tu primer envío en 5 minutos	2
2	Autenticación	3
3	Endpoints	4
4	Webhooks	9
5	Modo de prueba	11
6	Errores	12
7	Soporte	12

1 Tu primer envío en 5 minutos

Necesitás una API key (te la entrega GO EXPRESS, empieza con `ge_live_`). Con eso, este curl crea un envío real:

```
BASH

curl -X POST https://api.goexpressparaguay.com/api/v1/envios \
  -H "X-API-Key: ge_live_TU_KEY_ACA" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: pedido-12345" \
  -d '{
    "destinatarioNombre": "Maria Gonzalez",
    "destinatarioDireccion": "Av. Irrazabal 456 c/ Ruta 1",
    "destinatarioTelefono": "0971123456",
    "destinatarioCiudad": "Ciudad del Este",
    "destinatarioDepartamento": "Alto Parana",
    "peso": 2,
    "codigoReferencia": "PEDIDO-12345"
  }'
```

Respuesta (201):

```
JSON

{
  "trackingNumber": "GE2026001234",
  "codigoReferencia": "PEDIDO-12345",
  "estado": "pendiente",
  "origen": "Asunción",
  "destino": "Ciudad del Este",
  "costo": 30000,
  "costoSeguro": 0,
  "montoACobrar": 30000,
  "tipoPago": "anticipado",
  "fecha": "2026-08-12",
  "creadoEn": "2026-08-12T14:30:00.000Z"
}
```

Listo. El `trackingNumber` es tu identificador: con él consultás estado e historial. Cuatro cosas que conviene saber desde el arranque:

- **El precio lo calcula el servidor.** No mandás costo: se cotiza según la tarifa vigente de la ruta. Si mandás `costo`, se ignora.
- **El origen es tu ciudad registrada.** El envío sale desde la ciudad de tu cuenta de cliente.
- **Idempotency-Key te protege de duplicados.** Si tu sistema reintenta el POST (timeout, corte de red), el mismo envío se devuelve una sola vez en lugar de crearse dos veces. Usá un valor único por pedido, por ejemplo tu número de orden.
- **El tipo de pago lo elegís vos:** `anticipado` (default) o `contra_entrega`. Sin `tipoPago`, el envío es prepago y el `montoACobrar` que figura es la tarifa del flete más el seguro si corresponde, fijado por el servidor. Con `contra_entrega`, vos indicás cuánto cobra el repartidor al destinatario. Detalle en la sección de contra entrega, más abajo.

2 Autenticación

Todas las requests llevan tu key en el header `X-API-Key` :



- La key se muestra **una sola vez** al emitirse. Guardala en un gestor de secretos, nunca en el código fuente ni en el repositorio.
- Cada key pertenece a un cliente y solo opera sobre sus propios envíos.
- Las keys tienen permisos granulares: `crear_envios` , `consultar_envios` , `consultar_tarifas` , `webhooks` . Si una request responde 403, a tu key le falta el permiso de ese endpoint.
- Sin key o con key inválida la respuesta es 401.

Rotación de keys

Cuando necesites rotar (política de seguridad, sospecha de filtración, cambio de proveedor):

1. Pedile a GO EXPRESS la rotación. Se emite una key nueva y la anterior queda válida por una ventana acordada (48 horas por defecto).
2. Desplegá la key nueva en tu sistema dentro de esa ventana.
3. Al vencer la ventana, la key vieja deja de funcionar sola. Sin corte de servicio.

Si una key se compromete, GO EXPRESS puede revocarla en el acto (sin ventana).

3 Endpoints

POST

/api/v1/envios

Crea un envío. Permiso: `crear_envios`.

Campos del body:

CAMPO	TIPO	REQUERIDO	NOTAS
<code>destinatarioNombre</code>	string	Sí	
<code>destinatarioDireccion</code>	string	Sí	
<code>destinatarioTelefono</code>	string	Sí	Formato paraguayo, acepta <code>0971...</code> o <code>+595971...</code>
<code>destinatarioCiudad</code>	string	Sí	Debe tener tarifa configurada (ver GET /tarifas)
<code>destinatarioDepartamento</code>	string	No	
<code>peso</code>	number	Sí	En kg
<code>dimensiones</code>	objeto	No	{ "largo": 30, "ancho": 20, "alto": 10 } en cm, los tres juntos
<code>codigoReferencia</code>	string	No	Tu identificador interno (número de pedido)
<code>cantidad</code>	number	No	Default 1
<code>producto</code>	string	No	Descripción del contenido
<code>fragil</code>	boolean	No	Default false
<code>valorDeclarado</code>	number	No	En guaraníes, habilita seguro
<code>seguroAdicional</code>	boolean	No	Opt-in, el costo se calcula server-side
<code>instruccionesEntrega</code>	string	No	
<code>tipoPago</code>	string	No	<code>anticipado</code> (default) o <code>contra_entrega</code>
<code>montoACobrar</code>	number	Solo COD	En guaraníes. Requerido con <code>contra_entrega</code> , rechazado con <code>anticipado</code> (400)
<code>destinatarioTelefono2</code> , <code>destinatarioCedula</code> , <code>destinatarioBarrio</code> , <code>destinatarioReferencia</code> , <code>destinatarioEmail</code>	string	No	

Headers opcionales:

HEADER	USO
Idempotency-Key	8 a 128 caracteres (letras, números, guion, guion bajo). Un retry con la misma key devuelve el envío original con 200. La misma key con un body distinto responde 409.

Ejemplo JavaScript (fetch nativo):

```
JS

const res = await fetch('https://api.goexpressparaguay.com/api/v1/envios', {
  method: 'POST',
  headers: {
    'X-API-Key': process.env.GOEXPRESS_API_KEY,
    'Content-Type': 'application/json',
    'Idempotency-Key': `pedido-${orden.id}`,
  },
  body: JSON.stringify({
    destinatarioNombre: orden.cliente.nombre,
    destinatarioDireccion: orden.direccion,
    destinatarioTelefono: orden.telefono,
    destinatarioCiudad: orden.ciudad,
    peso: orden.pesoKg,
    codigoReferencia: `PEDIDO-${orden.id}`,
  }),
});

if (!res.ok) {
  const error = await res.json();
  throw new Error(`GO EXPRESS ${res.status}: ${error.code} ${error.error}`);
}

const envio = await res.json();
// envio.trackingNumber -> guardalo junto a tu orden
```

Envíos contra entrega (COD)

Por defecto todo envío creado por API es prepago. Si tu operación cobra al destinatario en la puerta, mandá `tipoPago: "contra_entrega"` junto con el monto que el repartidor debe cobrar:

```
BASH

curl -X POST https://api.goexpressparaguay.com/api/v1/envios \
  -H "X-API-Key: ge_live_TU_KEY_ACA" \
  -H "Content-Type: application/json" \
  -H "Idempotency-Key: pedido-67890" \
  -d '{
    "destinatarioNombre": "Carlos Benitez",
    "destinatarioDireccion": "Av. Monseñor Rodríguez 2345",
    "destinatarioTelefono": "0981555444",
    "destinatarioCiudad": "Ciudad del Este",
    "destinatarioDepartamento": "Alto Parana",
    "peso": 2,
    "codigoReferencia": "PEDIDO-67890",
    "tipoPago": "contra_entrega",
    "montoACobrar": 180000
  }'
```

Respuesta (201):

```
JSON

{
  "trackingNumber": "GE2026001235",
  "codigoReferencia": "PEDIDO-67890",
  "estado": "pendiente",
  "origen": "Asunción",
  "destino": "Ciudad del Este",
  "costo": 30000,
  "costoSeguro": 0,
  "montoACobrar": 180000,
  "tipoPago": "contra_entrega",
  "fecha": "2026-08-12",
  "creadoEn": "2026-08-12T15:10:00.000Z"
}
```

Cómo funciona el monto:

- `montoACobrar` es el total que el repartidor cobra al destinatario. Lo componés según tu modelo: si tu precio ya incluye el envío, es el precio de tu producto; si lo cobrás aparte, es mercadería más flete. GO EXPRESS retiene su tarifa (flete más seguro) al liquidar y te rinde el resto.
- El mínimo es la tarifa del envío. `montoACobrar` tiene que cubrir `costo + costoSeguro`. Si necesitás el número exacto antes de crear, cotizá con `GET /tarifas`.
- Si el monto no llega al mínimo, la respuesta es 422 `MONTO_INSUFICIENTE` y te dice el mínimo y cómo se compone:

JSON

```
{
  "error": "El montoACobrar (25000 Gs) no cubre la tarifa del envío. Mínimo: 30000 Gs (costo 30000 + seguro 0).",
  "code": "MONTO_INSUFICIENTE",
  "details": { "minimo": 30000, "costo": 30000, "costoSeguro": 0, "montoACobrar": 25000 }
}
```

- Con **anticipado** no mandes **montoACobrar** : responde 400. En prepago el monto lo fija el servidor, y aceptar el campo daría a entender que lo controlás.
- Nada cambia para las integraciones existentes. Sin **tipoPago** el envío sigue siendo **anticipado** , igual que siempre.

GET

/api/v1/envios/{trackingNumber}

Estado e historial completo de un envío tuyo. Permiso: **consultar_envios** .

BASH

```
curl https://api.goexpressparaguay.com/api/v1/envios/GE2026001234 \
-H "X-API-Key: ge_live_TU_KEY_ACA"
```

JSON

```
{
  "trackingNumber": "GE2026001234",
  "estado": "en_reparto",
  "eventos": [
    { "estado": "pendiente", "descripcion": "Envío creado via API", "ubicacion": null, "fecha": "2026-08-12T14:30:00.000Z" },
    { "estado": "recolectado", "descripcion": "Paquete recolectado", "ubicacion": "Asuncion", "fecha": "2026-08-12T18:00:00.000Z" },
    { "estado": "en_reparto", "descripcion": "En reparto final", "ubicacion": "Ciudad del Este", "fecha": "2026-08-13T09:15:00.000Z" }
  ]
}
```

Estados posibles: **pendiente** , **recolectado** , **en_transito** , **en_deposito** , **en_reparto** , **entregado** , **fallido** , **problema** .

Un tracking de otro cliente responde 404, igual que uno inexistente.

GET

/api/v1/envios

Listado paginado de tus envíos. Permiso: **consultar_envios** .

Query params: `page` (default 1), `limit` (default 20, máximo 100), `estado`, `fechaDesde`, `fechaHasta` (formato YYYY-MM-DD).

BASH

```
curl "https://api.goexpressparaguay.com/api/v1/envios?estado=entregado&fechaDesde=2026-08-01&limit=50" \
-H "X-API-Key: ge_live_TU_KEY_ACA"
```

La respuesta trae `data` (array de envíos) y `pagination` (`total`, `page`, `limit`, `totalPages`, `hasMore`).

GET

`/api/v1/tarifas`

Cotización de una ruta antes de crear el envío. Permiso: `consultar_tarifas` .

BASH

```
curl "https://api.goexpressparaguay.com/api/v1/tarifas?
origen=Asuncion&destino=Ciudad%20del%20Este&peso=2" \
-H "X-API-Key: ge_live_TU_KEY_ACA"
```

JSON

```
{ "matched": true, "costo": 30000, "moneda": "PYG", "origen": "Asunción", "destino": "Ciudad del Este" }
```

Si la ruta no tiene tarifa, `matched` es `false` y `costo` es `null` : el API nunca inventa precios. Params opcionales de dimensiones: `largo`, `ancho`, `alto` (los tres juntos, en cm).

4 Webhooks: enterate de cada cambio de estado

En lugar de consultar el estado cada X minutos, registrá una URL y GO EXPRESS te avisa con un POST cada vez que un envío tuyo cambia de estado.

Registrar tu endpoint

Con el permiso `webhooks` :

BASH

```
curl -X POST https://api.goexpressparaguay.com/api/v1/webhook-endpoints \
-H "X-API-Key: ge_live_TU_KEY_ACA" \
-H "Content-Type: application/json" \
-d '{ "url": "https://tusistema.com/hooks/goexpress" }'
```

JSON

```
{
  "id": "3f2b...",
  "url": "https://tusistema.com/hooks/goexpress",
  "eventos": ["envio.estado_cambiado"],
  "activo": true,
  "secreto": "whsec_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX",
  "aviso": "Guardá este secreto ahora: no se vuelve a mostrar."
}
```

La URL debe ser `https://` hacia un dominio público. El `secreto` firma cada delivery: guardalo junto a tu API key. `GET /api/v1/webhook-endpoints` lista tus endpoints y `DELETE /api/v1/webhook-endpoints/{id}` da de baja uno.

Qué recibís

Un POST con estos headers y body:

HTTP

```
X-GoExpress-Event: envio.estado_cambiado
X-GoExpress-Signature: sha256=3fd54a...
X-GoExpress-Delivery: 9b1c...
Content-Type: application/json
```

JSON

```
{
  "evento": "envio.estado_cambiado",
  "tracking": "GE2026001234",
  "estadoAnterior": "en_reparto",
  "estadoNuevo": "entregado",
  "codigoReferencia": "PEDIDO-12345",
  "timestamp": "2026-08-13T16:45:00.000Z"
}
```

Respondé con cualquier status 2xx en menos de 10 segundos. Procesá en background si tu lógica es pesada: primero el 200, después el trabajo.

Verificar la firma (obligatorio)

Cada POST viene firmado con HMAC-SHA256 sobre el body crudo, usando tu secreto. Verificala siempre antes de procesar: es lo que garantiza que el evento vino de GO EXPRESS. Snippet listo para pegar (Node.js/Express):

```
JS

const crypto = require('node:crypto');
const express = require('express');

const app = express();

app.post('/hooks/goexpress',
  express.raw({ type: 'application/json' })), // body CRUD0, sin parsear
(req, res) => {
  const firma = req.headers['x-goexpress-signature'] || '';
  const esperada = 'sha256=' + crypto
    .createHmac('sha256', process.env.GOEXPRESS_WEBHOOK_SECRET)
    .update(req.body) // req.body es un Buffer
    .digest('hex');

  const a = Buffer.from(firma);
  const b = Buffer.from(esperada);
  if (a.length !== b.length || !crypto.timingSafeEqual(a, b)) {
    return res.status(401).send('firma invalida');
  }

  const evento = JSON.parse(req.body.toString('utf8'));
  res.sendStatus(200); // ack primero

  // procesar evento.tracking / evento.estadoNuevo en background
}
);
```

Claves del snippet: la firma se calcula sobre los **bytes crudos** del body (si tu framework parsea el JSON antes, la verificación falla) y la comparación usa `timingSafeEqual`.

Reintentos

Si tu endpoint no responde 2xx, GO EXPRESS reintenta con espera creciente: al minuto, a los 5 minutos y a los 25 minutos. Tras el cuarto intento fallido el evento se descarta. Dos consecuencias prácticas:

- Diseñá tu handler **idempotente**: en casos raros un mismo evento puede llegar dos veces (usá `X-GoExpress-Delivery` para deduplicar).
- Si tu sistema estuvo caído más de media hora, reconciliá con `GET /api/v1/envios` al volver.

5 Modo de prueba: integrá sin tocar datos reales

Pedile a GO EXPRESS una key de prueba (empieza con `ge_test_`). Con ella desarrollás y probás toda la integración sin crear un solo envío real:

- **POST /envios** : valida tu payload y cotiza con las tarifas reales, pero no crea nada. Devuelve un envío simulado con tracking `GE-TEST-...` y `"simulated": true`. Funciona también con `contra_entrega` : errores de validación, `RUTA_SIN_TARIFA` y `MONTO_INSUFICIENTE` responden exactamente igual que en producción.
- **GET /envios** y **GET /envios/{tracking}** : devuelven 3 envíos de ejemplo fijos (`GE-TEST-0000000001` pendiente, `GE-TEST-0000000002` en reparto, `GE-TEST-0000000003` entregado, todos con `"simulated": true`) para que pruebes tu parseo de listados, detalle e historial de eventos.
- **GET /tarifas** : responde igual que en producción (cotizar no toca nada).
- **POST /test/webhook-event** : dispara un evento de muestra contra tu endpoint registrado, firmado con tu secreto real. Es la forma de probar tu verificación HMAC de punta a punta:

BASH

```
curl -X POST https://api.goexpressparaguay.com/api/v1/test/webhook-event \
-H "X-API-Key: ge_test_TU_KEY_DE_PRUEBA"
```

JSON

```
{
  "evento": "envio.estado_cambiado",
  "simulated": true,
  "resultados": [
    { "endpointId": "3f2b...", "url": "https://tusistema.com/hooks/goexpress", "entregado": true,
    "httpStatus": 200 }
  ]
}
```

Si `entregado` es `false`, revisá que tu endpoint responda 2xx y que la verificación de firma use el body crudo. El payload de prueba llega con `"simulated": true` para que puedas distinguirlo en tus logs.

Checklist de pase a producción: cuando el POST simulado, el parseo de los fixtures y el webhook de prueba funcionen, cambiá `ge_test_` por tu key `ge_live_` y no toques nada más. Los contratos son idénticos.

6 Errores

Todas las respuestas de error tienen la misma forma:

```
JSON
{ "error": "mensaje legible", "code": "CODIGO_ESTABLE", "details": { } }
```

Programá contra `code`, no contra el texto del mensaje.

HTTP	CODE	CUÁNDO	QUÉ HACER
400	<code>BAD_REQUEST</code>	Body o parámetros inválidos (detalle campo por campo en <code>details</code>)	Corregir el payload
401	<code>UNAUTHORIZED</code>	Key ausente, inválida, revocada o expirada	Revisar el header <code>X-API-Key</code> ; si rotaste, desplegar la key nueva
403	<code>FORBIDDEN</code>	A la key le falta el permiso del endpoint	Pedir el permiso a GO EXPRESS
404	<code>NOT_FOUND</code>	Tracking inexistente o de otro cliente	Verificar el tracking
409	<code>IDEMPOTENCY_KEY_REUSED</code>	Misma <code>Idempotency-Key</code> con un body distinto	Usar una key nueva por cada pedido distinto
409	<code>CONFLICT</code>	La <code>Idempotency-Key</code> corresponde a un envío anulado (la anulación la hace GO EXPRESS por canal operativo, no existe por API)	Usar una key nueva
422	<code>RUTA_SIN_TARIFA</code>	El destino no tiene tarifa configurada	Pre-validar con <code>GET /tarifas</code> ; contactar a GO EXPRESS para habilitar la ruta
422	<code>MONTO_INSUFICIENTE</code>	COD: <code>montoACobrar</code> no cubre <code>costo</code> + <code>costoSeguro</code>	Usar al menos el <code>mínimo</code> que viene en <code>details</code>
429	<code>TOO_MANY_REQUESTS</code>	Límite de 60 requests por minuto por key superado	Esperar y reintentar con backoff; los headers <code>RateLimit-*</code> indican cuándo
500	<code>INTERNAL_ERROR</code> / <code>DB_ERROR</code>	Error del lado de GO EXPRESS	Reintentar con backoff; si persiste, avisar

Recomendación general de reintentos: ante 429 o 5xx, reintentá con espera exponencial (1s, 2s, 4s...) y siempre con `Idempotency-Key` en los POST, así el reintento nunca duplica un envío.

7 Soporte

Cualquier duda de integración, rutas o permisos: escribinos a GO EXPRESS por el canal de siempre con tu `codigoReferencia` o `trackingNumber` a mano. Para reportar un problema del API, incluí el `X-Request-Id` que viene en los headers de cada respuesta: acelera muchísimo el diagnóstico.